

# Design Pattern

Design reusable solutions

# Design pattern overview

Design patterns are **standardized, reusable solutions** to commonly occurring problems in software design. They help create clean, scalable, and maintainable code by providing proven design templates. They do **not provide code**, but rather **proven structures** that guide how classes and objects interact.

## **Core Purposes of Design Patterns**

- **Reusable Solutions to Recurring Problems**

Provide proven solutions that can be reused across projects.

- **Improved Code Structure & Maintainability**

Organize code into clear, manageable, and maintainable components.

- **Enhanced Scalability, Flexibility & Reusability**

Enable systems to grow, adapt, and reuse components efficiently.

- **Standardized Design Practices**

Ensure consistent and predictable design across teams and projects.

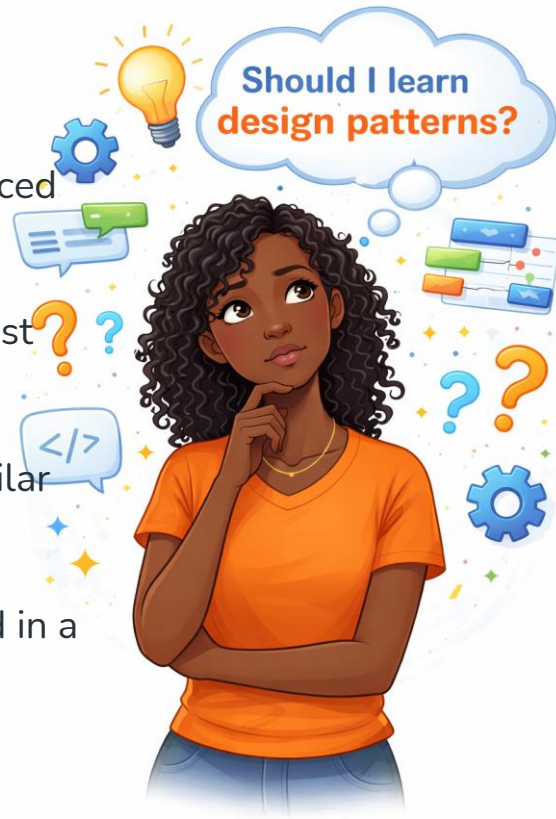
- **Clear Communication of Design Intent**

Provide a common vocabulary to understand and discuss system design.

# Why learning Design Patterns?

There are multiple reasons to learn design patterns:

- Code that is simpler to comprehend, update, and expand is produced with the help of design patterns.
- They offer solutions that have been tried and tested as well as best practices.
- Learning this enables them to quickly and effectively address similar challenges in various projects.
- Developers can produce reusable components that can be utilized in a variety of applications by implementing design patterns.
- This reduces redundancy and saves development time.



# Types of Design Pattern

# Creational Design Patterns



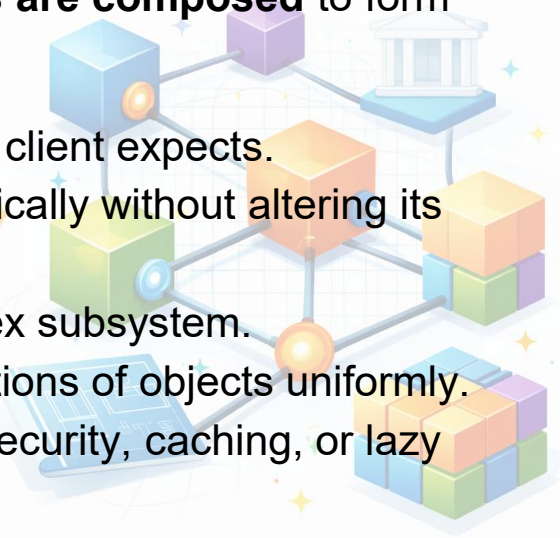
**Creational Design Patterns** deal with **object creation**, providing flexible and controlled ways to instantiate objects without tightly coupling code to specific classes.

- **Singleton Pattern:** Ensures only one instance of a class exists and provides a global access point to it.
- **Factory Method Pattern:** Defines an interface for creating objects, allowing subclasses to decide which class to instantiate.
- **Abstract Factory Pattern:** Creates families of related objects without specifying their concrete classes.
- **Builder Pattern:** Constructs complex objects step by step, separating construction from representation.
- **Prototype Pattern:** Creates new objects by copying an existing object rather than creating from scratch.
- **Object Pool Pattern:** Manages a pool of reusable objects to improve performance and resource usage.
- **Lazy Initialization:** Delays object creation until it is actually needed, improving efficiency.

## 2. Structural Design Patterns

**Structural Design Patterns** focus on **how classes and objects are composed** to form larger, flexible, and efficient structures.

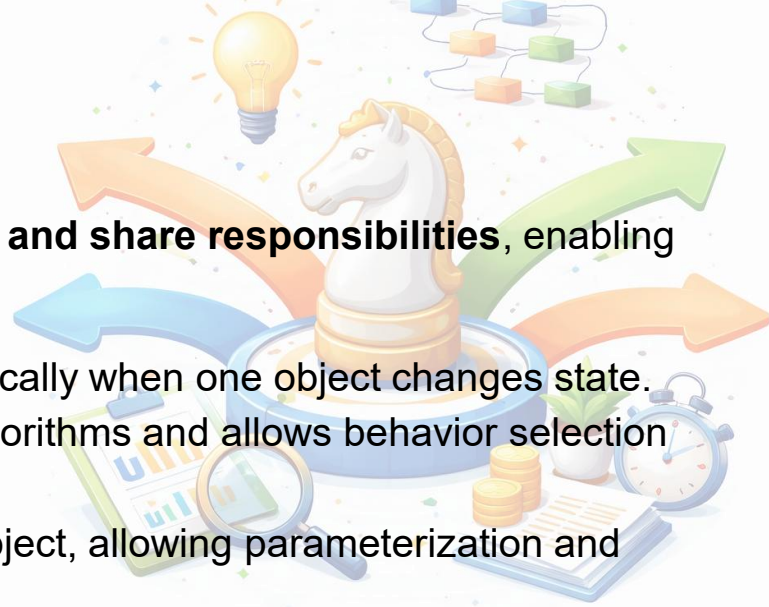
- **Adapter Pattern:** Converts one interface into another that a client expects.
- **Decorator Pattern:** Adds new behavior to an object dynamically without altering its structure.
- **Facade Pattern:** Provides a simplified interface to a complex subsystem.
- **Composite Pattern :** Treats individual objects and compositions of objects uniformly.
- **Proxy Pattern:** Controls access to another object, adding security, caching, or lazy loading.
- **Bridge Pattern:** Separates abstraction from implementation so both can vary independently.
- **Flyweight Pattern:** Reduces memory usage by sharing common object state.



### 3. Behavioral Design Patterns

**Behavioral Design Patterns** define **how objects interact and share responsibilities**, enabling flexible and maintainable communication.

- **Observer Pattern** : Notifies multiple objects automatically when one object changes state.
- **Strategy Pattern** : Encapsulates interchangeable algorithms and allows behavior selection at runtime.
- **Command Pattern** :Encapsulates a request as an object, allowing parameterization and undo operations.
- **Chain of Responsibility Pattern**: Passes a request in a chain of handlers until one processes it.
- **Template Method Pattern** : Defines the structure of an algorithm while allowing
- **Iterator Pattern** :Provides a standard way to traverse a collection without exposing its structure.
- **State Pattern**:Allows an object to change behavior when its internal state changes.
- Others:**Mediator Pattern, Memento Pattern, Visitor Pattern**





## 4. Advanced Design Patterns

**Advanced Design Patterns** focus on **enterprise-level architecture**, scalability, and long-term maintainability.

- **SOLID Principles:** Five principles that guide clean, maintainable, and scalable object-oriented design.
- **Dependency Injection Pattern:** Injects dependencies externally to reduce coupling and improve testability.
- **Composition vs Inheritance:** Encourages building behavior through composition rather than rigid inheritance.
- **Event-Driven Architecture & Patterns:** Systems react to events, improving scalability and loose coupling.
- **CQRS Pattern:** Separates read and write operations to optimize performance and scalability.
- **Event Sourcing Pattern:** Stores system state as a sequence of events rather than current values.
- **CQRS vs Event Sourcing:** CQRS focuses on command/query separation; Event Sourcing focuses on event-based state persistence.
- **Repository Pattern:** Abstracts data access logic, separating business logic from persistence.
- **MVC Design Pattern: Model–View–Controller (MVC):** separates an application into three interconnected components for clean architecture.



Popular design patterns explored

# 1. Singleton design Pattern

The Singleton Design Pattern ensures that a class has only one instance and provides a global access point to it. It is used when we want centralized control of resources, such as managing database connections, configuration settings or logging.

- Prevents accidental creation of multiple instances.
- Controls resource usage efficiently (e.g., memory, connections).
- Can simplify coordination across different parts of an application.



# Real World Applications

The Singleton Pattern ensures a single, globally accessible instance of a class, providing controlled access and consistent behavior across an application.

1. **Logging Systems** : Maintain a consistent logging mechanism across an application.
2. **Configuration Managers** : Centralize access to configuration settings.
3. **Database Connections** : Manage a single point of database access.
4. **Thread Pools** : Efficiently manage a pool of threads for concurrent tasks.

# Features

The Singleton Pattern ensures that a class has only one instance and provides a global point of access to it, while efficiently managing resources.

1. **Single Instance:** Ensures only one object of the class exists in the JVM.
2. **Global Access Point:** Provides a centralized way to access the instance.
3. **Lazy or Eager Initialization:** An Instance can be created at class load time (eager) or when first needed (lazy).
4. **Thread Safety:** Can be designed to work correctly in multithreaded environments.
5. **Resource Management:** Useful for managing shared resources like configurations, logging or database connections.
6. **Flexibility in Implementation:** Can be implemented using eager initialization, lazy initialization, double-checked locking or an inner static class.

# Singleton: Database Connection Manager

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;
    private DatabaseConnection() {
        try {
            String url = "jdbc:mysql://localhost:3306/mydb";
            String user = "root";
            String password = "password";

            connection = DriverManager.getConnection(url, user, password);
        } catch (SQLException e) {
            throw new RuntimeException("Database connection failed", e);
        }
    }
    public static synchronized DatabaseConnection getInstance() {
        if (instance == null) {
            instance = new DatabaseConnection();
        }
        return instance;
    }
    public Connection getConnection() {
        return connection;
    }
}
```

The Database Connection Manager is a common example of the **Singleton design pattern**, used to ensure that **only one database connection (or connection manager)** exists in an application. Creating multiple database connections is **resource-intensive** and can lead to performance issues or inconsistent data access.

By using Singleton, the connection manager class has a **private constructor**, preventing external instantiation. A **single static instance** is created and accessed through a **public static method**.

This guarantees that all parts of the application **share the same database connection**, improving **performance, consistency, and resource management**.

# Test usage

```
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.Statement;

public class MainApp {

    public static void main(String[] args) {

        try {
            // Get the singleton instance
            DatabaseConnection db = DatabaseConnection.getInstance();

            // Obtain the shared connection
            Connection connection = db.getConnection();

            // Use the connection (example query)
            Statement stmt = connection.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM users");

            while (rs.next()) {
                System.out.println(
                    rs.getInt("id") + " - " + rs.getString("name")
                );
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

`DatabaseConnection.getInstance()`

→ always returns **the same object**

`getConnection()`

→ provides **one shared database connection**

Multiple classes can reuse this connection safely

**No duplicate DB connections created**

The `main` method accesses the database through the Singleton

`DatabaseConnection` class, ensuring that all database operations use the same shared connection instance.

# Singleton example2

```
public class President {  
    private static President instance;  
    private String name;  
    // Private constructor  
    private President() {  
        this.name = "National President";  
    }  
    // Global access point  
    public static synchronized President getInstance() {  
        if (instance == null) {  
            instance = new President();  
        }  
        return instance;  
    }  
    public String makeDecision() {  
        return "Decision approved by the President.";  
    }  
    public String getName() {  
        return name;/  
    }  
}
```

```
public class PresidentServlet extends HttpServlet {  
  
    @Override  
    protected void doGet(HttpServletRequest request,  
        HttpServletResponse response)  
        throws ServletException, IOException {  
  
        // Access the Singleton President  
        President president = President.getInstance();  
  
        // Use the President object  
        String decision = president.makeDecision();  
  
        response.setContentType("text/plain");  
        response.getWriter().println(  
            president.getName() + ": " + decision  
        );  
    }  
}
```



# Example: Singleton 3

```
import org.hibernate.SessionFactory;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;
import org.hibernate.cfg.Configuration;
import org.hibernate.cfg.Environment;
import org.hibernate.service.ServiceRegistry;

import java.util.Properties;

public class HibernateUtil {

    private static SessionFactory sessionFactory;

    private HibernateUtil() {} // prevent instantiation

    public static synchronized SessionFactory getSessionFactory() {
        if (sessionFactory == null) {
            try {
                Configuration configuration = new Configuration();

                Properties settings = new Properties();
                settings.put(Environment.DRIVER, "com.mysql.cj.jdbc.Driver");
                settings.put(Environment.URL, "jdbc:mysql://localhost:3306/hibernating");
                settings.put(Environment.USER, "root");
                settings.put(Environment.PASS, "password");
                settings.put(Environment.DIALECT, "org.hibernate.dialect.MySQL8Dialect");

                settings.put(Environment.SHOW_SQL, "true");
                settings.put(Environment.HBM2DDL_AUTO, "update");

                configuration.setProperties(settings);

                // Register entity
                configuration.addAnnotatedClass(Student.class);

                ServiceRegistry serviceRegistry =
                    new StandardServiceRegistryBuilder()
                        .applySettings(configuration.getProperties())
                        .build();

                sessionFactory = configuration.buildSessionFactory(serviceRegistry);

            } catch (Exception e) {
                throw new RuntimeException("Failed to build SessionFactory", e);
            }
        }
        return sessionFactory;
    }
}
```

```
import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String firstName;

    @Column(nullable = false)
    private String lastName;

    public Student() {}

    public Student(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public Long getId() { return id; }
    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }

    public void setFirstName(String firstName) { this.firstName = firstName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
}
```

```
public class StudentService {

    private static StudentService instance;
    private final SessionFactory sessionFactory;
    private StudentService() {
        this.sessionFactory = HibernateUtil.getSessionFactory();
    }

    public static synchronized StudentService getInstance() {
        if (instance == null) {
            instance = new StudentService();
        }
        return instance;
    }

    public Student save(Student student) {
        Transaction tx = null;
        try (Session session = sessionFactory.openSession()) {
            tx = session.beginTransaction();
            session.persist(student);
            tx.commit();
            return student;
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            throw new RuntimeException("Save failed", e);
        }
    }

    public Student findById(Long id) {
        try (Session session = sessionFactory.openSession()) {
            return session.get(Student.class, id);
        }
    }

    public List<Student> findAll() {
        try (Session session = sessionFactory.openSession()) {
            return session.createQuery("FROM Student",
Student.class).list();
        }
    }
}
```

# Controller usage

```
@WebServlet("/student")
public class StudentController extends HttpServlet {

    private final StudentService studentService = StudentService.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        List<Student> students = studentService.findAll();
//or
        Student s = studentService.findById(id);

    }
}
```

# Real world example: Runtime



- Every running Java program has a Runtime object which represents the JVM
- It wouldn't make sense to have multiple instances of this since there can only be one JVM for a running program
- So it's accessed as a singleton, e.g.

```
public class Test {  
    public static void main(String[] args) {  
        Runtime r = Runtime.getRuntime();  
        System.out.println(r.freeMemory());  
    }  
}
```



# Factory design pattern

- The factory pattern uses an object instance to generate instances of another object
- The factory can return an instance of one of several possible depending on the data provided to it



# Factory example



- First we define the objects we will produce...

```
interface Car {  
    public int getTopSpeed();  
}  
  
class ToyotaCorolla implements Car {  
    public int getTopSpeed() { return 160; };  
}  
  
class Ferrari458 implements Car {  
    public int getTopSpeed() { return 325; };  
}
```

# Factory example



- Then create the factory object...

```
class CarFactory {  
    public Car createCar(boolean sports) {  
        if (sports)  
            return new Ferrari458();  
        else  
            return new ToyotaCorolla();  
    }  
}
```

```
CarFactory factory = new CarFactory();  
Car car1 = factory.createCar(true);
```

# Where to use factories...



- When a class can't anticipate which kind of class of object it must create
  - When you want to localize the knowledge of which class gets created
  - When you want to insulate the client from the actual type that is being instantiated
- 
- Often used in conjunction with the *strategy* pattern



# Where else to use factories...



- When you want to perform some initialization on objects before they are used, e.g.

```
class CarFactory {  
    public Car createCar(boolean sports) {  
        Car car = sports ? new Ferrari458() : new ToyotaCorolla();  
  
        car.checkOil();  
        car.fillWithPetrol();  
  
        return car;  
    }  
}
```

Our factory ensures  
that this initialization  
always occurs for new  
instances

# Real world example



```
import java.sql.*;

public class DatabaseConnectionTest {

    public static void main(String[] args) {
        try {
            Class.forName("oracle.jdbc.driver.OracleDriver");
            Connection con = DriverManager.getConnection("myServer",
                "user", "password");
            Statement stmt = con.createStatement();

        } catch (Exception e) {
        }
    }
}
```

# Observer pattern

- The observer pattern enables a publish /subscribe system
- Some object generate events, others are notified and then respond to them



# Observer pattern



- Objects can subscribe (register or attach) themselves to an event on another object
- They are the observers
- The pattern does not impose any limit to the number of observers that can subscribe a for a notification



# Observer example



- First we define the subject (publisher) – the object that will generate the events...

```
class Heart {  
    public void beat() {  
    }  
}
```

```
class HeartMonitor {  
    public void onBeat() {  
        System.out.print("__/\\__");  
    }  
}
```

# Observer example



- Then we add a mechanism for observers to attach themselves to the subject...

```
class Heart {  
    List<HeartMonitor> monitors  
        = new ArrayList<HeartMonitor>();  
  
    public void addMonitor(HeartMonitor monitor) {  
        monitors.add(monitor);  
    }  
  
    public void beat() {  
    }  
}
```

# Observer example



- Add make sure the observers are notified when the beat event occurs...

```
class Heart {  
    List<HeartMonitor> monitors  
        = new ArrayList<HeartMonitor>();  
  
    public void addMonitor(HeartMonitor monitor) {  
        monitors.add(monitor);  
    }  
  
    public void beat() {  
        for (HeartMonitor monitor : monitors)  
            monitor.onBeat();  
    }  
}
```

# Observer example



- We can now create a subject and attach some observers...

```
Heart heart = new Heart();
```

```
HeartMonitor monitor1 = new HeartMonitor();
```

```
HeartMonitor monitor2 = new HeartMonitor();
```

```
heart.addMonitor(monitor1);
```

```
heart.addMonitor(monitor2);
```

```
for (int i = 0; i < 100; i++)  
    heart.beat();
```

Both monitors will be  
notified



# Real world example: jQuery



- jQuery's event model uses functions as observers, e.g.

```
<input type="button" id="mybutton" value="Click!" />
```

```
function onMyButtonClick() {  
    alert('Clicked');  
}
```

```
$('#mybtn').click(onMyButtonClick);
```

The subject

The observer

Observers can  
be attached to  
the event on the  
subject

# Dependency Injection (DI)

The **Dependency Injection (DI) Design Pattern** is a design technique in which **an object's dependencies are provided from the outside** rather than being created internally by the object itself.

DI promotes **loose coupling, better testability, and cleaner code structure** by separating object creation from object usage.

# Why Dependency Injection (DI)

Dependency Injection is used when we want to:

- Reduce tight coupling between classes
- Improve flexibility and maintainability
- Make code easier to test and extend
- Follow SOLID principles (especially **Dependency Inversion Principle**)

DI allows applications to **change implementations without modifying dependent code.**

# Key Features of Dependency Injection

- **Loose Coupling** – Classes depend on abstractions, not implementations
- **Improved Testability** – Easy to inject mock objects
- **Better Maintainability** – Changes are localized
- **Reusable Components** – Same class can work with different dependencies
- **Clear Separation of Concerns** – Object creation is externalized

## Real World Applications of Dependency Injection

Dependency Injection is widely used in real-world Java applications:

- **Service Layers** – Injecting repositories into services
- **Web Applications** – Controllers receiving services
- **Frameworks** – Spring, Jakarta EE, Hibernate
- **Testing** – Injecting mock or fake implementations

DI is the foundation of **modern enterprise Java development**.

## Types of Dependency Injection

There are **three common types** of Dependency Injection:

1. **Constructor Injection** – Dependencies are provided through the constructor
2. **Setter Injection** – Dependencies are provided via setter methods
3. **Interface Injection** – Dependencies are provided through a dedicated interface

**Constructor Injection is the most recommended** in modern Java.

## Example without Dependency Injection

```
public class StudentService {  
    private StudentRepository repository = new  
    StudentRepository();  
  
    public void save() {  
        repository.save();  
    }  
}
```

Problems:

- Hard to test
- Repository implementation is fixed
- Tight coupling

## Example with Dependency Injection

```
public class StudentService {  
    private final StudentRepository repository;  
  
    public StudentService(StudentRepository repository) {  
        this.repository = repository;  
    }  
  
    public void save() {  
        repository.save();  
    }  
}
```

### Benefits

- Loose coupling
- Easy to replace implementation
- Easy to test



# Dependency Injection Using Interfaces

```
public interface StudentRepository {  
    void save();  
}
```

```
public class StudentRepositoryImpl implements StudentRepository {  
    public void save() {  
        System.out.println("Student saved");  
    }  
}  
  
StudentRepository repo = new StudentRepositoryImpl();  
StudentService service = new StudentService(repo);
```

DI allows **runtime flexibility** without changing business logic. DI allows **runtime flexibility** without changing business logic.

# Dependency injection vs Singleton

| <b>Dependency Injection</b>      | <b>Singleton</b>             |
|----------------------------------|------------------------------|
| Dependencies provided externally | Dependency accessed globally |
| Promotes loose coupling          | Can cause tight coupling     |
| Easy to test                     | Harder to mock               |
| Preferred in enterprise apps     | Use carefully                |

# Dependency injection Frameworks

Most Java frameworks use DI internally:

- **Spring Framework** – @Autowired, @Component
- **Jakarta EE** – @Inject
- **Hibernate** – Injected session factories
- **Testing Frameworks** – Mock injection

DI is the backbone of **modern Java frameworks**.

# Repository Design Pattern

- Definition:
- Encapsulates data access logic and provides a clean interface.
- Separates business logic from persistence logic.

# Why Use Repository Pattern?

- - Isolates database logic
- - Improves maintainability
- - Enhances testability
- - Supports multiple data sources

# Real World Applications

- - Enterprise Java Applications
- - Web Applications
- - ORM Frameworks
- - Microservices

# Key Features

- - Centralized data access
- - Loose coupling
- - Improved testability
- - Clean architecture
- - Scalability

# Repository Structure

- - Entity / Model
- - Repository Interface
- - Repository Implementation
- - Service Layer



# Without Repository Pattern

- ✗ Database logic mixed with business logic
- ✗ Hard to test
- ✗ Difficult to maintain

# Repository vs DAO

- Repository: Domain-focused
- DAO: Database-focused

# Framework Usage

- - Spring Data JPA
- - Hibernate
- - Jakarta EE
- - Microservices

# Example Without Repository Pattern (Bad Practice)

```
public class StudentService {  
    public void saveStudent(Student student) {  
        // Database logic inside service  
        // SQL / Hibernate code mixed with business logic  
    }  
}
```

Problems:

- Tight coupling
- Hard to test
- Difficult to maintain

# Repository Pattern Example (interface)

```
public interface StudentRepository {  
    void save(Student student);  
    Student findById(Long id);  
    List<Student> findAll();  
}
```

- Defines WHAT operations are allowed,
- not HOW they are implemented

# Repository Pattern Example (Implementation)

```
public class StudentRepositoryImpl implements StudentRepository {
```

```
    @Override
```

```
    public void save(Student student) {  
        System.out.println("Student saved");  
    }
```

```
    @Override
```

```
    public Student findById(Long id) {  
        return new Student();  
    }
```

```
    @Override
```

```
    public List<Student> findAll() {  
        return new ArrayList<>();  
    }
```

```
}
```

- Contains all persistence logic
- Encapsulates database operations

# Repository usage Service Layer

```
public class StudentService {  
    private final StudentRepository repository;  
  
    public StudentService(StudentRepository repository) {  
        this.repository = repository;  
    }  
  
    public void registerStudent(Student student) {  
        repository.save(student);  
    }  
}
```

# MVC Design Pattern

- Definition:
- Model–View–Controller (MVC) separates an application into three interconnected components to improve structure and maintainability.



# Why Use MVC?

- - Separates concerns
- - Improves maintainability
- - Supports parallel development
- - Enhances scalability

# MVC Components

- Model: Business logic and data
- View: User interface
- Controller: Handles user input

# Model

- Represents application data
- Contains business logic
- Independent of UI

# View

- Displays data to the user
- Handles presentation
- Receives updates from Model

# Controller

- Handles user input
- Updates the Model
- Selects appropriate View

# MVC Flow

A typical MVC interaction follows this sequence:

1. **User interacts with View** (clicks button / submits form)
2. **View sends request to Controller**
3. **Controller processes input** and calls Model logic
4. **Model updates data / returns result**
5. **Controller chooses View** (or returns data)
6. **View displays updated output to the user**

**Simple flow statement:**

**User → View → Controller → Model → View**

# Benefits of MVC

- **Cleaner architecture** (organized codebase)
- **Less coupling** (changes in UI won't break business logic easily)
- **Reusable components** (same Model can be used with different Views)
- **Easier debugging** (issues are easier to locate by layer)
- **Supports enterprise development** (standard in modern frameworks)

# MVC in Java Frameworks

MVC is common in Java-based applications such as:

- **Spring MVC / Spring Boot Web**
  - Controllers handle HTTP requests
  - Services/Models handle business logic
  - Views may be HTML templates (Thymeleaf/JSP) or JSON for APIs
- **JavaFX / Desktop Applications**
  - Controllers manage UI events
  - Model holds data/state
  - View is the UI layout (FXML or code-based UI)
- **Jakarta EE Applications**
  - Servlet/Controller layer
  - Business layer (EJB/services)
  - View layer (JSP/JSF)



# MVC vs Traditional Design

- MVC: Separated responsibilities
- Traditional: Mixed UI and logic

# Real World Usage

- Web applications
- Enterprise systems
- Desktop applications